

CLASA 10 C

Tablouri Unidimensionale (Vectori)

Concepte Fundamentale, Indexare și Operații de Bază

Sintaxă și Declarație

STRUCTURĂ ȘI SINTAXĂ

Un tablou unidimensional este o succesiune de elemente de **același tip**, alocate contiguu în memorie:

- ▶ Memoria alocată este continuă.
- ▶ Fiecare element este referit printr-un **indice**.
- ▶ Indicii sunt între 0 și Dimensiune - 1.

```
// Sintaxa generala:  
tipDeBaza nume[Dimensiune];  
  
// Exemple de declaratii:  
int X[10];  
int n, A[100], m, B[50];
```

Referirea Elementelor

OPERATORUL DE INDEXARE

Accesul la un element se realizează prin operatorul de indexare [], cu prioritate maximă:

- ▶ Un element indexat este tratat ca o variabilă obișnuită.
- ▶ **Atenție:** C++ nu verifică dacă indicele este în intervalul valid $[0, \text{Dim} - 1]$.
- ▶ Depășirea limitelor duce la **comportament nedefinit**.

```
int X[10];  
  
cin >> X[0];           // citire  
X[0] = 17;              //  
    atribuire  
cout << X[0];          // afisare  
cout << X[0] / 5;      // operatii
```

Valorile Inițiale ale Elementelor

GESTIUNEA MEMORIEI

Inițializarea implicită depinde direct de scopul declarării:

- ▶ **Tablou Global:** Elementele sunt garantate ca fiind inițializate cu **0**.
- ▶ **Tablou Local:** Elementele conțin valori reziduale / aleatorii (*garbage values*).

```
int A[100]; // Global: toate  
           sunt 0  
  
int main() {  
    int B[100]; // Local:  
               rezidual  
  
    // B[0] este impredictibil!  
    return 0;  
}
```

Dimensiune Fizică vs. Dimensiune Logică

CONCEPTE ESENȚIALE

- ▶ **Dimensiunea Fizică:** Numărul maxim de elemente rezervate în memorie la compilare (ex: 100).
- ▶ **Dimensiunea Logică (n):** Numărul curent de elemente utilizate efectiv în algoritm.
- ▶ **Regulă:** Trebuie respectată întotdeauna relația $n \leq \text{Dimensiune Fizică}$.

```
// Dimensiune fizica: 100
int X[100], n;

// Dimensiune logica: n
cin >> n;

for(int i = 0; i < n; i++) {
    X[i] = 1;
}
```

Citirea și Afișarea unui Vector

OPERAȚII DE BAZĂ

Tablourile unidimensionale nu se pot citi sau afișa dintr-o singură operație:

- ▶ `cin >> X;` generează o **eroare de sintaxă**.
- ▶ `cout << X;` va afișa **adresa de memorie** a primului element.
- ▶ Operațiile se realizează iterativ prin parcurgere.

```
cin >> n;  
// Citire  
for(int i = 0; i < n; i++)  
    cin >> X[i];  
  
// Afișare  
for(int i = 0; i < n; i++)  
    cout << X[i] << ' ';
```

Indexare de la 0 vs. Indexare de la 1

CONVENȚII ÎN PROBLEME

Multe enunțuri indexează datele începând de la 1:

- ▶ Putem ignora elementul $V[0]$.
- ▶ Elementele utilizate vor fi $V[1]$, ..., $V[n]$.
- ▶ **Important:** Dimensiunea fizică trebuie să fie cu cel puțin 1 mai mare decât limita maximă pentru n .

```
// Pentru  $n \leq 100$ :  
int V[101], n;  
  
cin >> n;  
for(int i = 1; i <= n; i++)  
    cin >> V[i];  
  
for(int i = 1; i <= n; i++)  
    cout << V[i] << ' ';
```

Inițializarea la Declarație

SINTAXĂ

Inițializarea directă este permisă **doar la declarație**:

- ▶ Elementele nespecificate se completează automat cu 0.
- ▶ `int A[10] = {1};` setează doar `A[0] = 1`, restul fiind 0.
- ▶ Reassignarea ulterioară cu acolade este interzisă sintactic.

```
int A[] = {10, 20, 30};  
// Dimensiune automata = 3  
  
int B[10] = {10, 20};  
// B[0]=10, B[1]=20, restul 0  
  
int C[10] = {0};  
// Toate sunt 0  
  
// ILEGAL ulterior:  
// C = {0}; // Eroare de  
// sintaxa!
```